

Aplicación de Patrones GOF en el caso de Búsqueda Biblioteca

Miguel Angel Zambrano Peña

Brayan Turmequé

Javier Alberto Ochoa Ávila

Patrones y Metodología de la Construcción del Software

Ingeniería de Software

Universidad Manuela Beltrán

2024

Introducción

El proyecto tiene como objetivo resolver el problema de diseñar un sistema eficiente para la búsqueda de libros en una biblioteca, considerando criterios variados como título, autor, género o ISBN. Este problema es relevante porque una búsqueda eficiente y modular no solo mejora la experiencia del usuario, sino que también permite escalar el sistema para manejar grandes volúmenes de datos en una biblioteca universitaria o pública.

La aplicación de patrones de diseño GOF ayuda a abordar este problema proporcionando soluciones estructuradas y probadas. Estos patrones aseguran que el sistema sea escalable, modular y fácil de mantener, permitiendo agregar nuevas funcionalidades o algoritmos de búsqueda con un impacto mínimo en la arquitectura existente. Por ejemplo, con el **Strategy**, se puede implementar y cambiar fácilmente entre diferentes algoritmos de búsqueda según las necesidades, mientras que el **Composite** facilita la organización jerárquica de los libros en diferentes categorías, mejorando la navegabilidad del sistema.

Descripción del problema

El sistema debe satisfacer la necesidad de los usuarios de buscar libros en una biblioteca por diferentes criterios como título, autor, género o ISBN. Actualmente, las búsquedas son ineficientes, carecen de modularidad y dificultan la escalabilidad al incorporar nuevos criterios o manejar grandes volúmenes de datos.

Los desafíos principales incluyen:

- Diseñar un sistema flexible que permita incorporar fácilmente nuevos algoritmos de búsqueda.
- Garantizar la eficiencia en tiempo de respuesta, incluso con bases de datos grandes.
- Facilitar la navegación a través de una estructura jerárquica de categorías de libros.

Alcance

Incluye:

1. Implementación de búsquedas por título, autor, género y ISBN.
2. Generación de resultados en tiempo real.
3. Creación de una interfaz para agregar nuevos libros al sistema.
4. Notificaciones a los usuarios sobre cambios relevantes, como la disponibilidad de libros.

Excluye:

1. Funcionalidades relacionadas con el préstamo o reserva de libros.
2. Integración con sistemas externos, como bibliotecas digitales o catálogos globales.
3. Gestión de usuarios y roles dentro del sistema.

Funcionales:

- Permitir búsquedas por título, autor, género y ISBN.
- Mostrar resultados en tiempo real.
- Ofrecer una interfaz para la adición de nuevos libros.

No Funcionales:

- Escalabilidad para grandes volúmenes de datos.
- Modularidad para facilitar el mantenimiento.
- Eficiencia en las consultas.

Selección de Patrones de Diseño

Lista de patrones aplicados

1. **Factory Method:** Para manejar la creación de criterios de búsqueda (por título, autor, género, ISBN).
2. **Composite:** Para organizar los libros en una jerarquía de categorías y subcategorías, facilitando búsquedas estructuradas.
3. **Observer:** Para notificar a los usuarios sobre cambios en el sistema, como la disponibilidad de un libro buscado.
4. **Strategy:** Para permitir diferentes algoritmos de búsqueda (lineal, binaria, por índices optimizados, etc.).

Explicación de la elección

1. Factory Method

Este patrón permite crear objetos de búsqueda basados en diferentes criterios (título, autor, género, ISBN) sin especificar explícitamente sus clases. Es ideal para manejar los diferentes tipos de búsquedas de manera extensible y escalable.

Mejora del sistema: Facilita la adición de nuevos criterios de búsqueda sin alterar el código existente, promoviendo la modularidad.

2. Composite

Este patrón se utiliza para representar la jerarquía de categorías de libros, como géneros (ficción, no ficción) y subgéneros (misterio, ciencia ficción).

Mejora del sistema: Simplifica la navegación por categorías y permite tratar las colecciones de libros de manera uniforme, ya sea un grupo o un libro individual.

3. Observer

Permite que los usuarios interesados sean notificados automáticamente cuando un libro de su interés se vuelve disponible.

Mejora del sistema: Automatiza la comunicación con los usuarios, mejorando la experiencia y reduciendo la carga manual en la biblioteca.

4. Strategy

Implementa diferentes algoritmos de búsqueda, como búsquedas lineales para bases de datos pequeñas y búsquedas binarias o indexadas para bases de datos grandes.

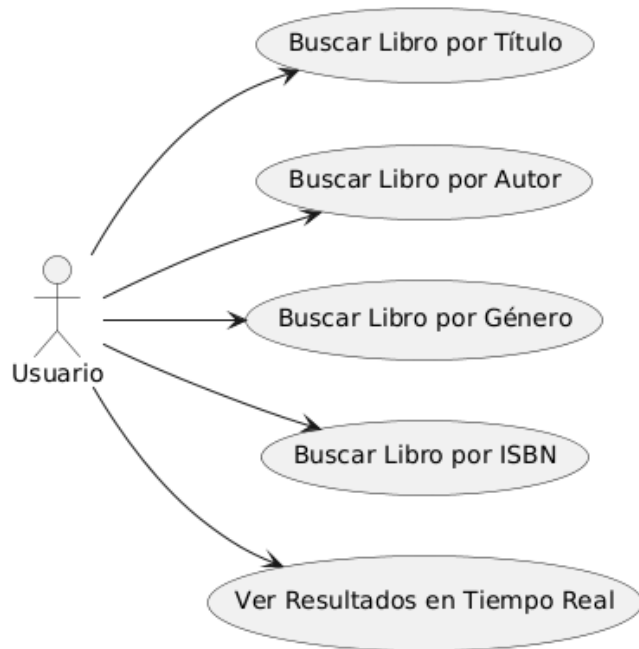
Mejora del sistema: Proporciona flexibilidad para elegir el algoritmo más eficiente según las características del sistema, mejorando el rendimiento de las búsquedas.

Diseño del Sistema

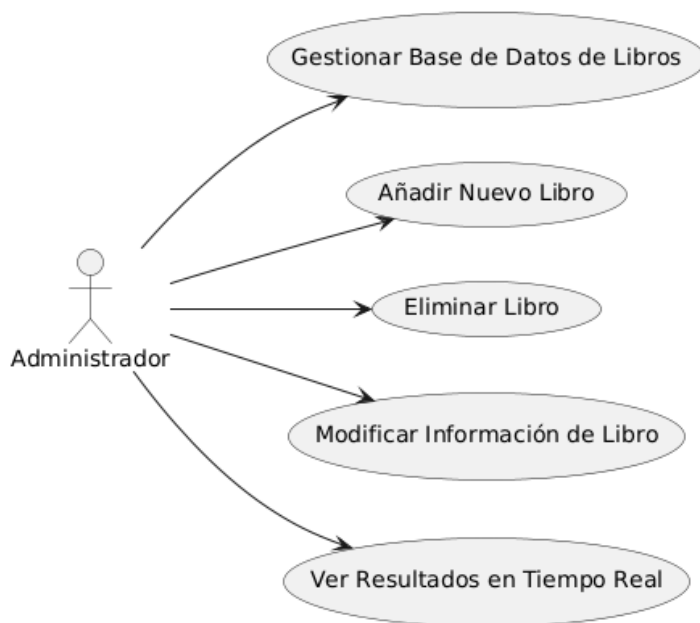
Diagramas UML:

Diagrama de casos de uso:

Usuarios:



Administrador:



Bibliotecario:

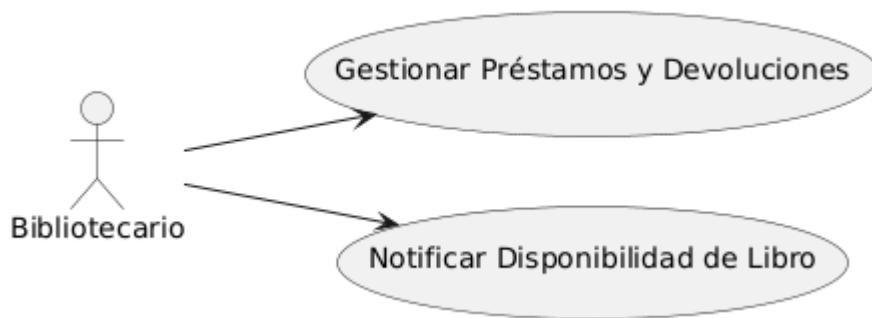


Diagrama de clases:

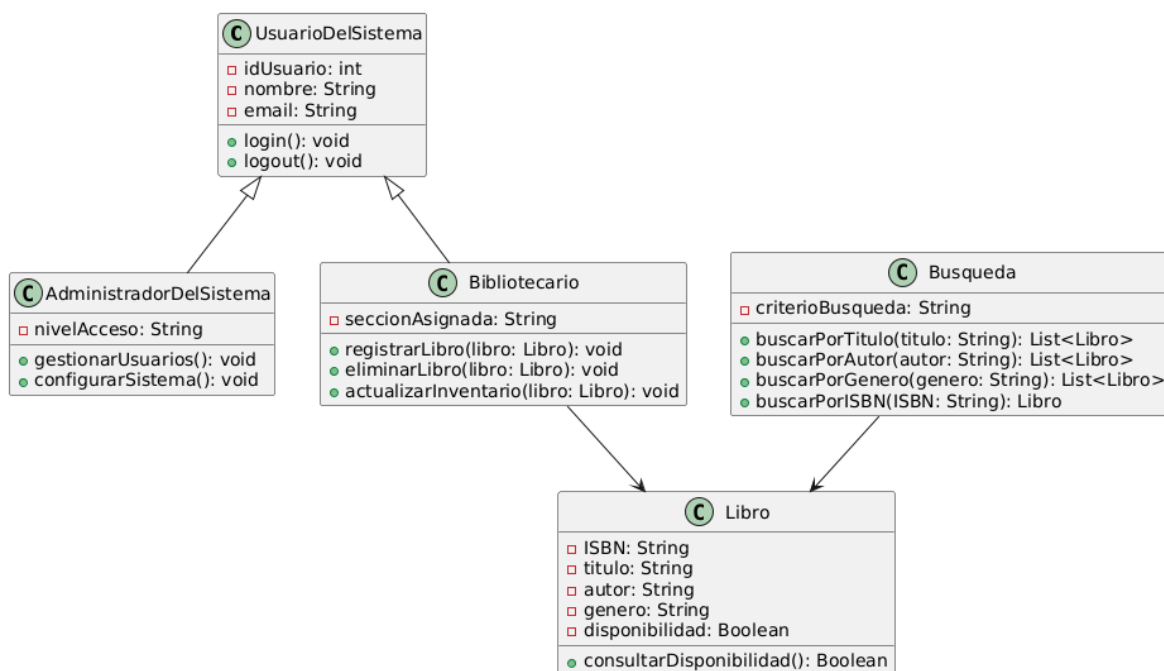


Diagrama de secuencia:

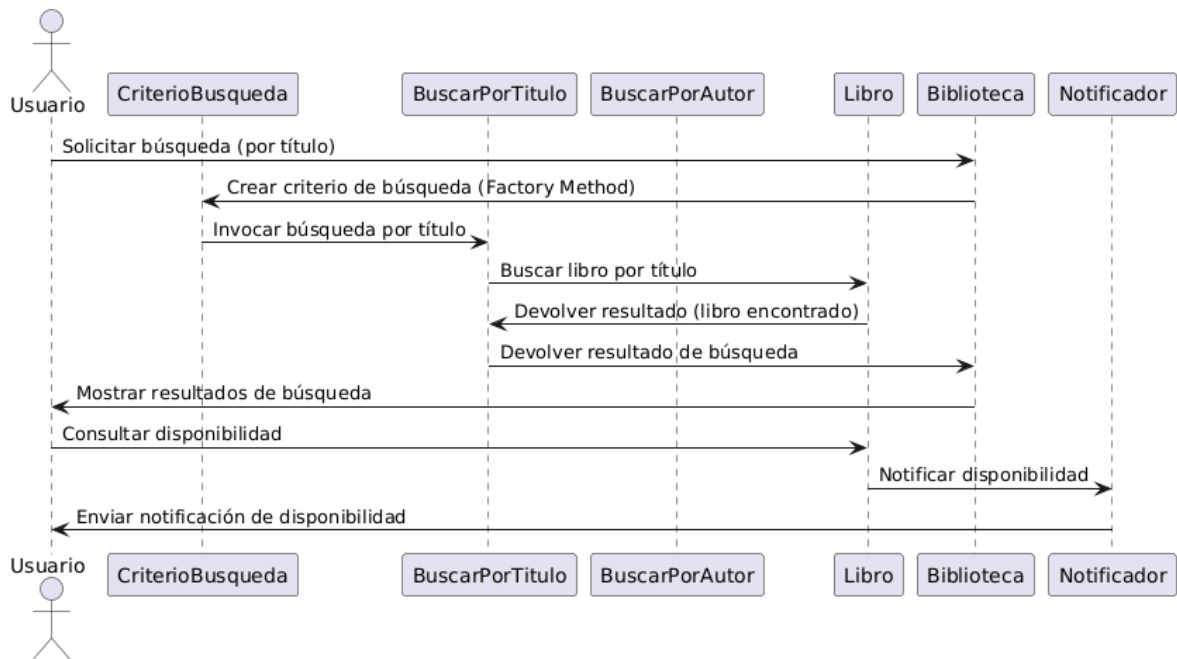
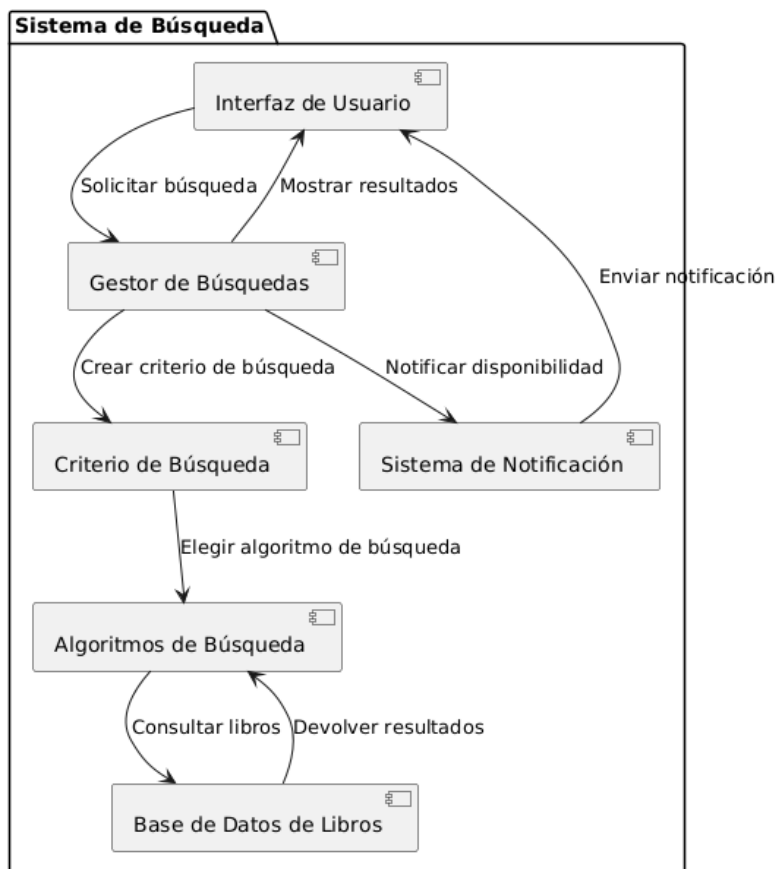
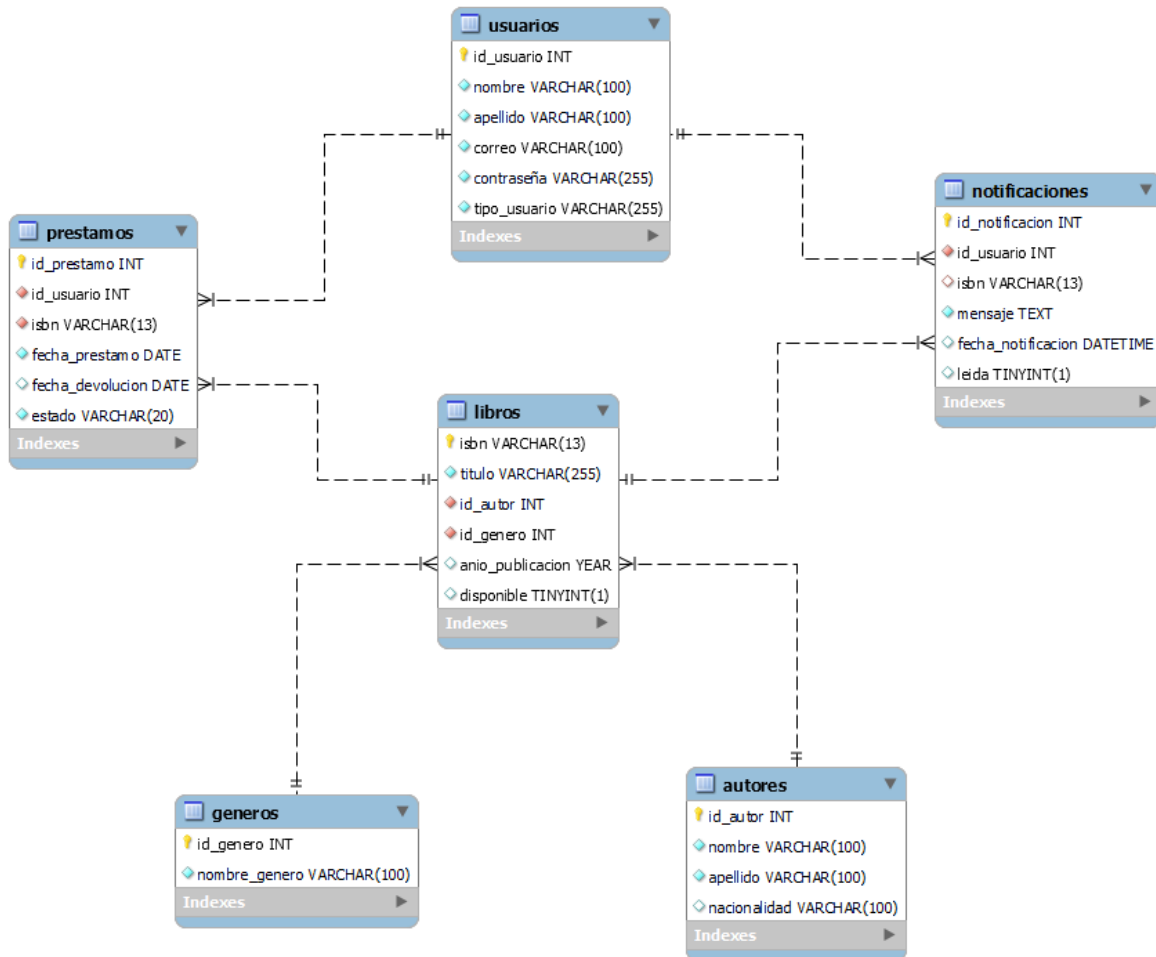


Diagrama de componentes:



Modelo Relacional:



Justificación del Diseño

Modularidad

El uso de patrones como Factory Method y Strategy permitió dividir el sistema en componentes específicos para cada funcionalidad. Por ejemplo, la creación de criterios de búsqueda está encapsulada en el Factory Method, lo que evita que el código principal dependa directamente de las implementaciones concretas de cada criterio. Esto hace que el sistema sea más fácil de modificar y expandir, ya que agregar un nuevo criterio o algoritmo de búsqueda no afecta el resto del código.

Escalabilidad

El Composite fue clave para organizar los libros en una estructura jerárquica, lo que facilita la búsqueda y navegación, incluso cuando se manejan grandes volúmenes de datos. Por otro lado, el Strategy permite implementar algoritmos más eficientes a medida que

crece la base de datos, asegurando que el sistema pueda responder rápidamente, sin importar el tamaño de la biblioteca.

Mantenimiento

Al separar las responsabilidades mediante patrones como Observer, es más sencillo realizar actualizaciones en el sistema. Por ejemplo, cualquier cambio en la lógica de notificaciones no afecta la lógica de búsqueda ni la estructura de los datos. Además, el uso de patrones bien definidos facilita que otros desarrolladores comprendan y trabajen en el sistema, reduciendo el esfuerzo necesario para corregir errores o añadir funcionalidades.

Ventajas de Separar las Responsabilidades

Separar las responsabilidades asegura que cada parte del sistema tenga una función clara y específica. Esto reduce el riesgo de errores, ya que los cambios en una parte del sistema no afectan a otras. Por ejemplo:

1. Búsquedas y notificaciones independientes: El Observer se encarga únicamente de notificar a los usuarios sobre cambios, mientras que los algoritmos de búsqueda se manejan por el Strategy, permitiendo que cada funcionalidad sea desarrollada y mantenida de manera aislada.
2. Fácil extensibilidad: Al tener criterios de búsqueda encapsulados en el Factory Method, agregar un nuevo criterio, como "año de publicación", no implica modificar el código existente.
3. Legibilidad y trabajo en equipo: La separación de responsabilidades hace que el diseño sea más intuitivo y fácil de entender, lo que facilita el trabajo colaborativo y reduce el tiempo de incorporación para nuevos desarrolladores.

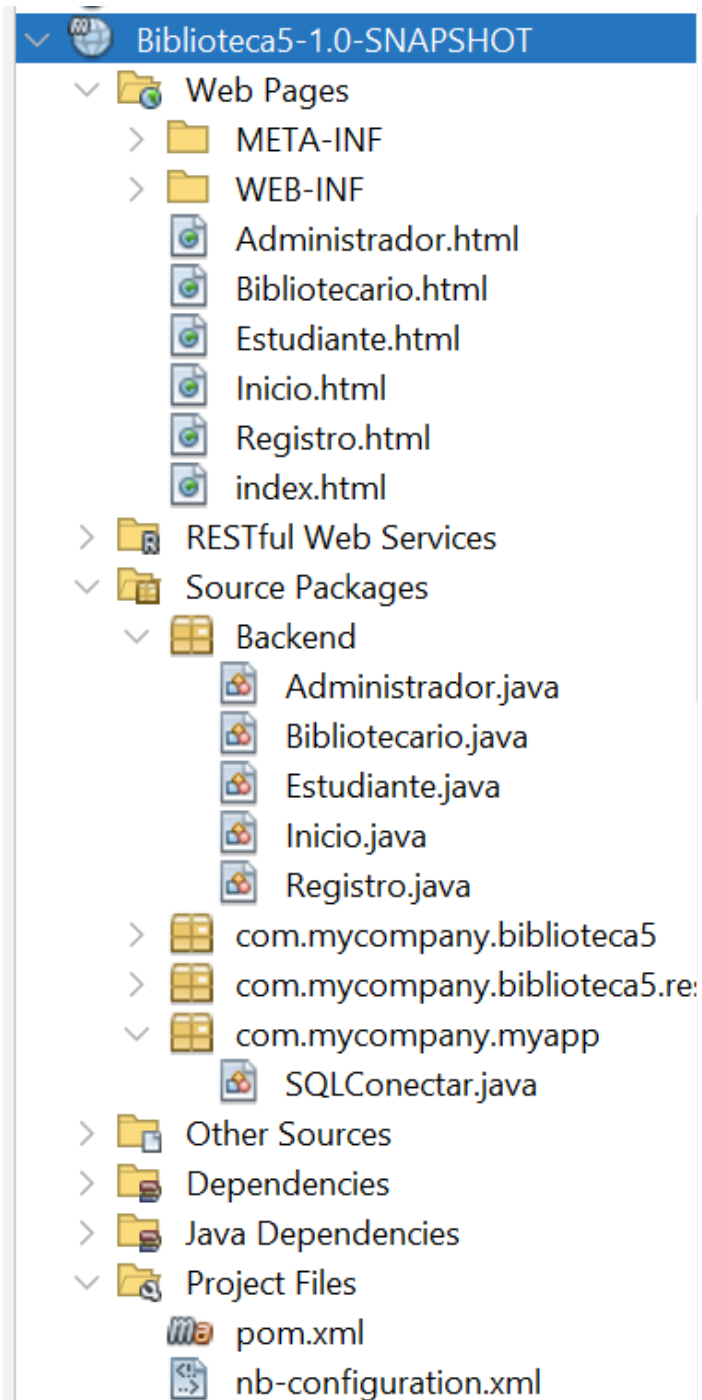
En resumen, la implementación de estos patrones no solo resolvió los problemas del caso de uso, sino que también dejó un sistema modular, escalable y fácil de mantener, listo para futuros desafíos y expansiones.

Implementación

Lenguajes y herramientas utilizadas:

Java, JavaScript y html

Estructura del código:



Sql-Script:

CREATE DATABASE Biblioteca;

USE Biblioteca;

CREATE TABLE Autores (

id_autor INT AUTO_INCREMENT PRIMARY KEY,

nombre VARCHAR(100) NOT NULL,

apellido VARCHAR(100) NOT NULL,

nacionalidad VARCHAR(100)

);

CREATE TABLE Generos (

id_genero INT AUTO_INCREMENT PRIMARY KEY,

nombre_genero VARCHAR(100) NOT NULL UNIQUE

);

CREATE TABLE Libros (

isbn VARCHAR(13) PRIMARY KEY,

titulo VARCHAR(255) NOT NULL,

id_autor INT NOT NULL,

id_genero INT NOT NULL,

anio_publicacion YEAR,

disponible BOOLEAN DEFAULT TRUE,

CONSTRAINT fk_autor FOREIGN KEY (id_autor) REFERENCES Autores(id_autor) ON DELETE CASCADE ON UPDATE CASCADE,

CONSTRAINT fk_genero FOREIGN KEY (id_genero) REFERENCES Generos(id_genero) ON DELETE CASCADE ON UPDATE CASCADE

);

```
CREATE TABLE Usuarios (  
id_usuario INT AUTO_INCREMENT PRIMARY KEY,  
nombre VARCHAR(100) NOT NULL,  
apellido VARCHAR(100) NOT NULL,  
correo VARCHAR(100) UNIQUE NOT NULL  
);
```

```
ALTER TABLE Usuarios  
ADD contraseña VARCHAR(255) NOT NULL;
```

```
ALTER TABLE Usuarios  
ADD tipo_usuario varchar(255)NOT NULL;
```

```
ALTER TABLE Usuarios  
DROP COLUMN fecha_registro;
```

```
CREATE TABLE Prestamos (  
id_prestamo INT AUTO_INCREMENT PRIMARY KEY,  
id_usuario INT NOT NULL,  
isbn VARCHAR(13) NOT NULL,  
fecha_prestamo DATE NOT NULL,  
fecha_devolucion DATE,  
CONSTRAINT fk_usuario FOREIGN KEY (id_usuario) REFERENCES  
Usuarios(id_usuario) ON DELETE CASCADE ON UPDATE CASCADE,  
CONSTRAINT fk_libro FOREIGN KEY (isbn) REFERENCES Libros(isbn) ON  
DELETE CASCADE ON UPDATE CASCADE  
);  
ALTER TABLE Prestamos  
ADD COLUMN estado VARCHAR(20) NOT NULL DEFAULT 'Pendiente';
```

```

CREATE TABLE Notificaciones (
    id_notificacion INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT NOT NULL,
    isbn VARCHAR(13),
    mensaje TEXT NOT NULL,
    fecha_notificacion DATETIME DEFAULT CURRENT_TIMESTAMP,
    leida BOOLEAN DEFAULT FALSE,
    CONSTRAINT fk_usuario_notificacion FOREIGN KEY (id_usuario)
REFERENCES Usuarios(id_usuario) ON DELETE CASCADE ON UPDATE
CASCADE,
    CONSTRAINT fk_libro_notificacion FOREIGN KEY (isbn) REFERENCES
Libros(isbn) ON DELETE SET NULL ON UPDATE CASCADE
);

SHOW TABLES;

```

	Tables_in_biblioteca
►	autores
	generos
	libros
	notificaciones
	prestamos
	usuarios

Fragmentos Destacados:

```

// Estrategia para búsquedas
interface SearchStrategy {
    List<Book> search(String criteria, Connection connection) throws Exception;
}

```

```

// Implementación de Strategy para búsqueda por título
class TitleSearchStrategy implements SearchStrategy {

    @Override
    public List<Book> search(String criteria, Connection connection) throws Exception {
        List<Book> results = new ArrayList<>();
        String query = "SELECT titulo, CONCAT(Autores.nombre, ' ', Autores.apellido) AS autor, Generos.nombre_genero AS genero, Libros.isbn, Libros.anio_publicacion "
            + "FROM Libros "
            + "JOIN Autores ON Libros.id_autor = Autores.id_autor "
            + "JOIN Generos ON Libros.id_genero = Generos.id_genero "
            + "WHERE titulo LIKE ?";

        try (PreparedStatement stmt = connection.prepareStatement(sql: query)) {
            stmt.setString(parameterIndex: 1, "%" + criteria + "%");
            try (ResultSet rs = stmt.executeQuery()) {
                while (rs.next()) {
                    results.add(new Book(titulo: rs.getString(columnLabel: "titulo"), autor: rs.getString(columnLabel: "autor"), genero: rs.getString(columnLabel: "genero"), isbn: rs.getString(columnLabel: "isbn"), anio_publicacion: rs.getString(columnLabel: "anio_publicacion")));
                }
            }
        }

        return results;
    }
}

```

```

// Implementación de Strategy para búsqueda por autor
class AuthorSearchStrategy implements SearchStrategy {

    @Override
    public List<Book> search(String criteria, Connection connection) throws Exception {
        List<Book> results = new ArrayList<>();
        String query = "SELECT titulo, CONCAT(Autores.nombre, ' ', Autores.apellido) AS autor, Generos.nombre_genero AS genero, Libros.isbn, Libros.anio_publicacion "
            + "FROM Libros "
            + "JOIN Autores ON Libros.id_autor = Autores.id_autor "
            + "JOIN Generos ON Libros.id_genero = Generos.id_genero "
            + "WHERE CONCAT(Autores.nombre, ' ', Autores.apellido) LIKE ?";

        try (PreparedStatement stmt = connection.prepareStatement(sql: query)) {
            stmt.setString(parameterIndex: 1, "%" + criteria + "%");
            try (ResultSet rs = stmt.executeQuery()) {
                while (rs.next()) {
                    results.add(new Book(titulo: rs.getString(columnLabel: "titulo"), autor: rs.getString(columnLabel: "autor"), genero: rs.getString(columnLabel: "genero"), isbn: rs.getString(columnLabel: "isbn"), anio_publicacion: rs.getString(columnLabel: "anio_publicacion")));
                }
            }
        }

        return results;
    }
}

```

```

// Implementación de Strategy para búsqueda por género
class GenreSearchStrategy implements SearchStrategy {

    @Override
    public List<Book> search(String criteria, Connection connection) throws Exception {
        List<Book> results = new ArrayList<>();
        String query = "SELECT titulo, CONCAT(Autores.nombre, ' ', Autores.apellido) AS autor, Generos.nombre_genero AS genero, Libros.isbn, Libros.anio_publicacion "
            + "FROM Libros "
            + "JOIN Autores ON Libros.id_autor = Autores.id_autor "
            + "JOIN Generos ON Libros.id_genero = Generos.id_genero "
            + "WHERE Generos.nombre_genero LIKE ?";

        try (PreparedStatement stmt = connection.prepareStatement(sql: query)) {
            stmt.setString(parameterIndex: 1, "%" + criteria + "%");
            try (ResultSet rs = stmt.executeQuery()) {
                while (rs.next()) {
                    results.add(new Book(titulo: rs.getString(columnLabel: "titulo"), autor: rs.getString(columnLabel: "autor"), genero: rs.getString(columnLabel: "genero"), isbn: rs.getString(columnLabel: "isbn"), anio_publicacion: rs.getString(columnLabel: "anio_publicacion")));
                }
            }
        }

        return results;
    }
}

```

```

// Factory Method para crear estrategias
class SearchStrategyFactory {

    public static SearchStrategy getStrategy(String type) {
        switch (type.toLowerCase()) {
            case "titulo":
                return new TitleSearchStrategy();
            case "autor":
                return new AuthorSearchStrategy();
            case "genero":
                return new GenreSearchStrategy();
            default:
                throw new IllegalArgumentException("Tipo de búsqueda no soportado: " + type);
        }
    }
}

```

Resultados

Programa:

Pagina Inicial:

¡Bienvenido a la Biblioteca!

Por favor, selecciona tu tipo de usuario para continuar:

Administrador

Gestiona todo el sistema.

Ingresar

Bibliotecario

Administra los libros.

Ingresar

Estudiante

Busca y reserva libros.

Ingresar

¿Aún no tienes una cuenta?

Registrarse

Registro:

Registro de Usuario

Por favor, completa los siguientes campos para registrarte.

Nombre:

Apellido:

Correo Electrónico:

Contraseña:

Tipo de Usuario:

Selecciona un tipo de cuenta

Registrarse

Inicio de Sesión

Por favor, ingresa tus credenciales para continuar.

Correo Electrónico:

Contraseña:

[Iniciar Sesión](#)

¿No tienes una cuenta? [Regístrate aquí](#)

Cerrar Sesión

Gestión de Biblioteca

Lista de Libros

Título	Autor	Nacionalidad	ISBN	Género	Año
5555	Miguel Angel Zambrano Pella	colombia	2222	drama	1999
5555	Miguel Angel Zambrano Pella	colombia	3333	drama	1999
5555	Miguel zzz	colombia	4444	drama	1999
El Principito	Antoine de Saint-Exupéry	Frances	9785478887194	Juvenil	1943
Z	pape l	colombia	9999	accion	1999

Agregar/Modificar Libro

Título:

El Principito

Nombre del Autor:

Antoine

Apellido del Autor:

Apellido del autor

Nacionalidad del Autor:

Frances

ISBN:

9785478887194

Género:

Juvenil

Año:

1943

Agregar

Modificar

Eliminar

Pagina Estudiante:

Ingrese su ID de usuario para cargar sus solicitudes:

13

Aceptar

Cancelar

Buscar Libros

Género

juvenil

Buscar

Título	Autor	Género	ISBN	Año	Acción
El Principito	Antoine de Saint-Exupéry	Juvenil	9788478887194	1943	Solicitar

Solicitud realizada exitosamente.

Aceptar

Pagina Bibliotecario:

Libros Disponibles

TÍTULO	AUTOR	GÉNERO	ISBN	AÑO	ACCIÓN
El Principito	Antoine de Saint-Exupéry	Juvenil	9788478887194	1943	Aceptar Rechazar

Solicitud realizada exitosamente.

Aceptar

Solicitud Denegada exitosamente.

Aceptar

Resultados Funcionales

1. Permitir búsquedas por diferentes criterios: Los usuarios pueden buscar libros por título, autor, género.
2. Generación de resultados en tiempo real: Se asegura una respuesta inmediata a las consultas realizadas.
3. Interfaz para agregar libros: Existe una funcionalidad que permite a los administradores incluir nuevos libros en el sistema.

Resultados No Funcionales

1. Eficiencia: Se buscó optimizar las consultas para que funcionen bien incluso con grandes volúmenes de datos.
2. Modularidad: El sistema está diseñado para facilitar el mantenimiento, permitiendo agregar nuevos criterios de búsqueda o funcionalidades sin afectar la estructura existente.
3. Escalabilidad: Puede manejar grandes cantidades de información, asegurando un rendimiento adecuado a medida que crece la base de datos.

Además, se utilizaron patrones de diseño como Factory Method, Composite, Strategy para garantizar estas características no funcionales, logrando que el sistema sea fácil de expandir y mantener. Si necesitas más detalles o análisis sobre algún punto específico, házmelo saber.

Evaluación Comparativa del Sistema: Con vs Sin Patrones de Diseño

Aspecto	Sin Patrones de Diseño	Con Patrones de Diseño
Escalabilidad	Difícil de expandir sin alterar código existente.	Fácil incorporación de nuevas funcionalidades gracias a Factory Method y Composite.
Mantenimiento	Altamente acoplado, difícil de actualizar.	Modular y fácil de mantener; responsabilidades claramente separadas.
Eficiencia	Algoritmos rígidos, bajo rendimiento en bases grandes.	Optimización dinámica mediante Strategy; mejor rendimiento.

Conclusiones

Reflexión sobre el aprendizaje

Al trabajar en la implementación de los patrones de diseño, ocurrieron varios desafíos. Uno de los más grandes fue entender cómo seleccionar el patrón adecuado para cada problema. Al principio, algunos patrones parecían similares o aplicables a múltiples situaciones, lo que generaba dudas sobre cuál elegir. Además, pasar de la teoría a la práctica fue complicado, especialmente con patrones como Composite, donde diseñar una jerarquía clara de clases requería bastante planeación. Otro reto importante fue integrar diferentes patrones, como Strategy y Factory Method, sin que el código se volviera confuso o poco eficiente.

Sin embargo, había muchos beneficios al usar patrones de diseño. El sistema quedó más estructurado y modular, lo que facilitó agregar nuevas funcionalidades sin tener que reescribir grandes partes del código. También fue evidente cómo estos patrones mejoraron la escalabilidad del sistema, permitiéndome incorporar fácilmente nuevos criterios de búsqueda o algoritmos. Finalmente, trabajar con patrones estandarizados hizo que el código fuera más legible, lo que sería de gran ayuda si alguien más necesita trabajar en el proyecto.

Mejoras futuras

En el futuro, creo que se podría optimizar varios aspectos del sistema. Por ejemplo, explorar algoritmos más avanzados para la búsqueda, como aquellos basados en inteligencia artificial, para hacer el sistema aún más eficiente. También sería interesante conectar el sistema con bases de datos externas o catálogos globales, algo que actualmente está fuera del alcance.

Además, creo que se debe mejorar la documentación de los patrones que implementé. Esto no solo sería útil para otros desarrolladores, sino también para mí, ya que serviría como referencia al expandir o modificar el sistema. También se necesita trabajar en mejorar la interfaz de usuario, añadiendo funcionalidades como sugerencias en tiempo real al escribir en la barra de búsqueda, aprovechando patrones como Observer para hacerla más dinámica.

En general, implementar patrones de diseño ayuda a comprender mejor cómo abordar problemas complejos en el desarrollo de software.

Bibliografía

https://sedici.unlp.edu.ar/bitstream/handle/10915/21723/Documento_completo.pdf?sequence=1

<https://www.scielo.cl/pdf/infotec/v24n3/art12.pdf>

<https://www.infor.uva.es/~felix/datos/priii/tema7.pdf>

Código GitHub:

https://github.com/Miguecapi/Final_Patrones